Menangani Input dari Client

Pemrograman Web • 14 slides • Generated on 11 October 2026, 22:42

Mahasiswa belajar menerima data dari pengguna melalui URL maupun form.

Web Programming #4

Slide 1

- **Materi:**
  - Query string: `url.Values` dan konversi tipe
  - Form HTML: `action`, `method`, `name`
  - GET dan POST
  - Membaca body form: `r.ParseForm()`, `r.PostFormValue()`
  - Escape output, validasi di server, menampilkan error
  - Pola Post/Redirect/Get

Query String: "url.Values"

Slide 2

- `r.URL.Query()` mem-parse `?q=golang&tag=web&tag=go` menjadi `url.Values`
- `url.Values` = `map[string][]string`, satu key bisa punya banyak nilai

```
q := r.URL.Query()
q.Get("q")      // "golang" — nilai pertama, "" kalau tidak ada
q.Has("hal")    // true kalau dikirim, walau nilainya kosong
q["tag"]        // []string{"web", "go"}
```

- `%20` dan `+` di URL akan di-decode jadi spasi

Input selalu dalam bentuk string

```
hal, err := strconv.Atoi(r.URL.Query().Get("hal"))
if err != nil || hal < 1 {
    hal = 1     // nilai default
}
```

Form HTML

Slide 3

```
<form action="/cari" method="GET">
    <label>Kata kunci <input type="text" name="q"></label>
    <button type="submit">Cari</button>
</form>
```

- Tiga atribut dalam form HTML:
  - `action` — path tujuan. Kosong = path halaman ini
  - `method` — `GET` atau `POST`. Default `GET`
  - `name` di tiap input — jadi **key** yang dibaca server
- Yang dikirim: pasangan `name=value` dari tiap input yang punya `name`
- Input tanpa `name` → tidak ikut terkirim
- Tipe input yang sering dipakai:

    - `text`, `email`, `password`, `number`
    - `<textarea>` — teks panjang
    - `<select>` — pilihan tunggal
    - `checkbox` dengan `name` sama — pilihan ganda
- `required`, `type="email"`, `maxlength` = validasi di **browser**
    - Bagus untuk UX, **tidak cukup** untuk keamanan

GET vs POST

Slide 4

	GET	POST
Data dikirim lewat	URL (query string)	body request
Terlihat di address bar	ya	tidak
Bisa di-bookmark / dibagikan	ya	tidak
Refresh halaman	langsung bisa	browser ask "kirim ulang?"

```
GET /cari?q=golang HTTP/1.1
Host: localhost:8080

(tanpa body)
```

```
POST /tamu HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded

nama=Andi&pesan=Halo
```

Form GET: Pencarian

Slide 5

```
const formCari = `<!DOCTYPE html>
<html lang="id">
<head><meta charset="utf-8"><title>Pencarian</title></head>
<body>
  <form action="/cari" method="GET">
    <input type="text" name="q" placeholder="Kata kunci...">
    <button type="submit">Cari</button>
  </form>
</body>
</html>`

func formCariHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    fmt.Fprint(w, formCari)
}

func cariHandler(w http.ResponseWriter, r *http.Request) {
    q := strings.TrimSpace(r.URL.Query().Get("q"))
    if q == "" {
        http.Redirect(w, r, "/", http.StatusSeeOther)   // balik ke form
        return
    }
    fmt.Fprintf(w, "Mencari: %s\n", q)
```

```
}
```

```
mux.HandleFunc("GET /{$}", formCariHandler)
mux.HandleFunc("GET /cari", cariHandler)
```

- Buka `/` → ketik `golang web` → klik Cari → browser membuka `/cari?q=golang+web`
- Form GET **hanya menyusun URL**, sama saja dengan mengetik URL secara manual

Membaca Body POST: "ParseForm"

Slide 6

- Body form dikirim dengan `Content-Type: application/x-www-form-urlencoded`
- Formatnya sama dengan query string (`nama=Andi&pesan=Halo`), hanya tempatnya yang berbeda (di body request)

```
func simpanHandler(w http.ResponseWriter, r *http.Request) {
    err := r.ParseForm()
    if err != nil {
        http.Error(w, "form tidak valid", http.StatusBadRequest)
        return
    }
    nama := r.PostFormValue("nama") // ambil single value
    hobi := r.PostForm["hobi"]      // checkbox: []string -> return slice
}
```

- `ParseForm()` membaca body **sekali**, hasilnya disimpan di:
  - `r.PostForm` — hanya dari body
  - `r.Form` — body **digabung** query string

`FormValue` vs `PostFormValue`

- `r.FormValue("nama")` — cari di body, lalu di query string
  - Praktis
  - Field yang tidak ada di body bisa **diisi lewat URL**: `POST /daftar?role=admin`
- `r.PostFormValue("nama")` — hanya dari body

Menampilkan Input dari User

Slide 7

- Setelah membaca input, misal kita tampilkan:

```
fmt.Fprintf(w, "<h1>Halo, %s!</h1>", nama)
```

- Coba isi nama dengan:

```
<script>alert('Hello')</script>
```

- Browser **menjalankan** script itu. Ini **XSS** (Cross-Site Scripting)
- User bisa kirim script yang bisa mencuri cookie/session **user lain**

`EscapeString` sebelum masuk HTML

```
import "html"

fmt.Fprintf(w, "<h1>Halo, %s!</h1>", html.EscapeString(nama))
```

```
// tag <script> tampil sebagai teks, tidak dieksekusi
```

- `html.EscapeString` mengganti `< > & ' "` menjadi entitas HTML
- Aturan: **setiap** nilai dari user yang masuk ke HTML wajib di-escape
  - Termasuk `value="..."` di input saat form ditampilkan ulang

> *Di materi selanjutnya, kita pakai `html/template` sehingga escape terjadi **otomatis**.*

Validasi di Server

Slide 8

- Validasi di server **perlu** dilakukan
- Contoh beberapa yang biasanya diperiksa:

1. **Rapikan** — `strings.TrimSpace`
2. **Kosong** — field wajib harus ada isinya
3. **Panjang** — batas minimal **dan maksimal**
4. **Format** — email, angka, tanggal

```
import (
   "unicode/utf8"
   "strings"
)

nama := strings.TrimSpace(r.PostFormValue("nama"))

if nama == "" {
```

```
    // "Nama wajib diisi"
}
if utf8.RuneCountInString(nama) > 50 {
    // "Nama maksimal 50 karakter"
}
```

- `len(nama)` menghitung **byte**, bukan karakter
    - `len("Müller")` = 7, `len("??")` = 6
    - Hitung karakter: `utf8.RuneCountInString`
- Cek format email apakah valid atau tidak:

```
import "net/mail"

if _, err := mail.ParseAddress(email); err != nil {
    // "Format email tidak valid"
}
```

Pengumpulan Error

Slide 9

- Pola tidak disarankan: berhenti di error pertama → user perbaiki, submit, dapat error berikutnya
- Lebih baik: periksa **semua** field, kumpulkan pesannya per field

```
func validasiTamu(nama, pesan string) map[string]string {
    errs := map[string]string{}

    if nama == "" {
        errs["nama"] = "Nama wajib diisi"
    } else if utf8.RuneCountInString(nama) > 50 {
        errs["nama"] = "Nama maksimal 50 karakter"
    }

    if utf8.RuneCountInString(pesan) < 5 {
        errs["pesan"] = "Pesan minimal 5 karakter"
    }

    return errs
}
```

- Di handler:

```
if errs := validasiTamu(nama, pesan); len(errs) > 0 {
    // tampilkan form lagi
    return
}
// lolos ? simpan
```

Setelah Submit Form

Slide 10

Gagal validasi → form tampil lagi

- Kembalikan ke form dengan populate isian sebelumnya

```
w.Header().Set("Content-Type", "text/html; charset=utf-8")
w.WriteHeader(http.StatusUnprocessableEntity)
fmt.Fprintf(w, `<input name="nama" value="%s"> <small>%s</small>`,
    html.EscapeString(nama), errs["nama"])
```

Sukses → redirect, jangan langsung render

```
simpan(tamu)
http.Redirect(w, r, "/tamu", http.StatusSeeOther)   // 303
```

- Pola **Post/Redirect/Get**:

```
POST /tamu   ?  303 See Other, Location: /tamu
GET  /tamu   ?  200, halaman berisi data baru
```

Data Bersama Antar Request

Slide 11

- Kode sample hari ini menyimpan tamu di slice global (belum pakai database)
- **tiap request = goroutine sendiri**
- Kalau ada dua orang submit bersamaan maka dua goroutine `append` ke slice yang sama, sehingga **bisa menyebabkan data race**

```
import "sync"

var (
    mu         sync.Mutex
    daftarTamu []Tamu
)

// menulis
mu.Lock()
daftarTamu = append(daftarTamu, tamu)
mu.Unlock()

// membaca: salin dulu, lepas kunci, baru render
mu.Lock()
salinan := slices.Clone(daftarTamu)
mu.Unlock()
```

- `Mutex` membuat hanya satu goroutine yang bisa masuk, yang lain antre
- Pegang kunci **sesingkat mungkin** (jangan menulis response sambil memegang kunci)
- Go bisa mendeteksi race: `go run -race main.go`

Buku Tamu (1/3) — Data & Validasi

Slide 12

```
mkdir buku-tamu && cd buku-tamu
go mod init buku-tamu
```

```
package main

import (
    "fmt"
    "html"
    "log"
    "net/http"
    "slices"
    "strings"
    "sync"
    "time"
    "unicode/utf8"
)

type Tamu struct {
    Nama  string
    Pesan string
    Waktu time.Time
}
```

```
var (
    mu         sync.Mutex
    daftarTamu []Tamu
)

func validasiTamu(nama, pesan string) map[string]string {
    errs := map[string]string{}

    if nama == "" {
        errs["nama"] = "Nama wajib diisi"
    } else if utf8.RuneCountInString(nama) > 50 {
        errs["nama"] = "Nama maksimal 50 karakter"
    }

    if utf8.RuneCountInString(pesan) < 5 {
        errs["pesan"] = "Pesan minimal 5 karakter"
    } else if utf8.RuneCountInString(pesan) > 500 {
        errs["pesan"] = "Pesan maksimal 500 karakter"
    }

    return errs
}
```

Buku Tamu (2/3) — Tampilan

Slide 13

```
const halamanHTML = `<!DOCTYPE html>
<html lang="id">
<head><meta charset="utf-8"><title>Buku Tamu</title></head>
<body>
  <h1>Buku Tamu</h1>
  <form method="POST" action="/">
    <p>Nama: <input name="nama" value="%s"> <small>%s</small></p>
    <p>Pesan: <textarea name="pesan">%s</textarea> <small>%s</small></p>
    <button type="submit">Kirim</button>
  </form>
  <hr>
%s
</body>
</html>`

func daftarHTML(tamu []Tamu) string {
    if len(tamu) == 0 {
        return "<p>Belum ada tamu.</p>"
    }
    var b strings.Builder
    for i := len(tamu) - 1; i >= 0; i-- {        // terbaru di atas
        t := tamu[i]
        fmt.Fprintf(&b, "<p><strong>%s</strong> <small>%s</small><br>%s</p>\n",
```

```
            html.EscapeString(t.Nama),
            t.Waktu.Format("02 Jan 15:04"),
            html.EscapeString(t.Pesan))
    }
    return b.String()
}

func renderHalaman(w http.ResponseWriter, status int, nama, pesan string, errs map[string]string) {
    mu.Lock()
    tamu := slices.Clone(daftarTamu)
    mu.Unlock()

    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    w.WriteHeader(status)
    fmt.Fprintf(w, halamanHTML,
        html.EscapeString(nama), errs["nama"],
        html.EscapeString(pesan), errs["pesan"],
        daftarHTML(tamu))
}
```

Buku Tamu (3/3) — Handler & main

Slide 14

```
func formHandler(w http.ResponseWriter, r *http.Request) {
    renderHalaman(w, http.StatusOK, "", "", nil)
}

func simpanHandler(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 64<<10)   // maks 64 KB
    if err := r.ParseForm(); err != nil {
        http.Error(w, "form tidak valid", http.StatusBadRequest)
        return
    }

    nama := strings.TrimSpace(r.PostFormValue("nama"))
    pesan := strings.TrimSpace(r.PostFormValue("pesan"))

    if errs := validasiTamu(nama, pesan); len(errs) > 0 {
        renderHalaman(w, http.StatusUnprocessableEntity, nama, pesan, errs)
        return
    }

    mu.Lock()
    daftarTamu = append(daftarTamu, Tamu{Nama: nama, Pesan: pesan, Waktu: time.Now()})
    mu.Unlock()
```

```
    http.Redirect(w, r, "/", http.StatusSeeOther)
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /{$}", formHandler)
    mux.HandleFunc("POST /{$}", simpanHandler)

    log.Println("Server jalan di http://localhost:8080")
    log.Fatal(http.ListenAndServe(":8080", mux))
}
```

- Jalankan: `go run main.go`